MADRE Agentic System

· Model-Agnostic · Delayed · Reasoning · Effort · [Beta v0.1]

Architecture Description

Scvria Sofware

June 20, 2026

Contents

Conceptual Architecture

1.1 Formal Requirements

This chapter states architectural requirements at product level.

Requirement	Architectural meaning
Local-first operation	The kernel must be able to run useful workflows on local user-owned infrastructure before optional remote integrations are considered.
Model agnosticism	All model execution enters through `ModelAction`, allowed `ModelBinding`, replaceable model-use profile, and model backend contracts, never through a vendor-specific core design.
Delayed reasoning	The runtime must separate immediate conversation from deeper work that needs decomposition, evidence, verification, internal actions, or more compute.
Context governance	Context must be selected, minimized, classified, provenanced, scoped, and revocable before it reaches models or internal actions.
Bounded internal actions	Executable behavior must be declared, policy-checked, traceable, and constrained by side effect and recovery rules. Typed runtime-defined internal actions are the canonical low-risk action form.
Record/knowledge isolation	retained interaction material, `KnowledgeRecord`, untrusted source material, `KnowledgeCandidate`, and `LearningCandidate` must not collapse into one undifferentiated prompt store.
Learning control	A `LearningCandidate` is an evaluated proposed improvement and changes active module behavior only through a defined promotion boundary; storing a `KnowledgeRecord` is not learning.
Inspectable execution	The CLI, local developer console, `RuntimeJournal`, and read models must make runtime behavior understandable without bypassing policy.

1.2 High-Level Architecture

MADRE is organized around a deterministic runtime kernel that coordinates governed reasoning modules through responsibility boundaries rather than through a fixed topology.

The central architectural rule is simple: the model is never the system. Models are invoked through `ModelAction` under context, policy, action, knowledge, and trace constraints. Each concrete `MADREAgent`

is owned by one `ReasoningModule` and constrained by `ModelBinding`. Agents may be short-lived, persistent, specialized, conservative, speculative, or workflow-bound, but they do not own the kernel or carry module knowledge across ownership boundaries.

The kernel coordinates modules; it does not become the reasoning actor for every task. System-level reasoning belongs in governed modules. Kernel registration assigns one compatible `ReasoningModule` the Core Module role for incoming interaction, continuity, clarification, planning, delegation, and delayed-work decisions. Within that module, one module-owned `SystemAgent` holds the Main Agent role; other `ComplexAgent` objects may exist without holding that role.

1.2.1 Conceptual Components

Component	Responsibility
Access surfaces	Present the system through CLI, local read-first inspection console, graphical UI, speech, or domain applications without becoming the source of runtime authority.
Core Module	A kernel-assigned `ReasoningModule` role that maintains interaction continuity, clarifies intent, creates reasoning plans, delegates to modules, and decides when deeper work is needed.
Runtime kernel	Coordinates module registration, routing, scheduling, resource limits, policies, action registries, durable records, recovery, and lifecycle.
Reasoning modules	Execute bounded reasoning domains through module-owned agents, workflows, routines, actions, records, policies, known-material boundaries, and allowed model/inference scope.
Context and knowledge	Provides scoped context bundles and `KnowledgeRecord` objects with provenance, scope, sensitivity, intended use, truth metadata, and correction paths.
Local Model Runtime Boundary	Product-level boundary for local or explicitly authorized inference through `ModelBinding`, model-use profile, model backend, `RuntimeEvent` evidence, and generated material.
Agent Action boundary	Exposes typed, bounded internal actions with declared inputs, outputs, side-effect class, policy requirements, trace evidence, and recovery expectations. External actions are exceptional risk boundaries, not the default execution path.
Learning and evolution	Evaluates `LearningCandidate` objects and promotes only authorized module improvements; known-material storage is separate.
Observability and recovery	Gives developers and authorized users a read-first view of `ReasoningTask`, `RuntimeEvent`, `PolicyDecision`, failures, and rollback options through `RuntimeJournal`.

1.2.2 Responsibility Layer Diagram

Figure 1.1 presents the primary responsibility structure of **MADRE** . The diagram is not a sequential request pipeline. It describes the ownership boundaries that constrain all runtime behavior.

The **MADRE** runtime kernel is the deterministic system coordinator. It owns access-surface entry, module registration, Core Module assignment, module routing, scheduling, resource assignment, global policy boundaries, `RuntimeJournal` coordination, durable runtime state, and lifecycle boundaries. The kernel does not become the reasoning actor for ordinary tasks. It selects, activates, limits, and observes reasoning modules.

Reasoning modules are the runtime units that perform domain-bounded reasoning. The module assigned the Core Module role is the first receiver for ordinary user interaction because its registered capability covers interaction continuity, clarification, general planning, and delegation. Other modules may specialize in research, knowledge handling, code, media, resource optimization, or domain-specific work. A module may answer quickly, create delayed work, invoke internal actions, or evaluate outcomes when its local policy and assigned resources allow it.

Executable behavior is not exposed as arbitrary external tools. The normal execution path uses an Agent Action boundary composed of typed runtime-defined functions with declared inputs, outputs, side-effect class, policy requirements, `RuntimeEvent` trace obligations, and recovery expectations. The first reference implementation may implement these actions on a JVM runtime, but JVM binding is not an architecture-level authority. External services, host commands, provider-backed tools, remote model services, or arbitrary plugin surfaces are not part of the default path and require stronger authority boundaries.

The Local Model Runtime Boundary is useful product-level phrasing for the point below the module and action layer where allowed inference executes. It is realized through `ModelBinding`, model-use profile, model backend, `RuntimeEvent` evidence, and generated material; it is not a chatbot server or an authority. Generated material returns to the module as generated material and is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself.

1.2.3 Concurrent Agent Lanes

MADRE separates immediate interaction from deeper reasoning, but this separation is not modeled as two global subsystems. It is a possible execution pattern for a module-owned `MADREAgent`.

A `MADREAgent` with a `ComplexAgent` profile may maintain three concurrent lanes. The foreground lane preserves conversational continuity, answers only when safe known context is sufficient, asks for clarification when intent or risk is unclear, and creates deferred work when more evidence is required. The reasoning lane performs planning, decomposition, evidence gathering, action invocation, verification, and result construction under module authority. The reflection lane reviews recent work, observes delayed outcomes, detects failure patterns, and proposes evaluated `LearningCandidate` objects.

This pattern may exist in any reasoning module. The Core Module may use it for general interaction and delegation. A research module may use it for evidence comparison and verification. A knowledge module may use it for extraction, provenance checks, correction, and deletion semantics. The kernel schedules and constrains these lanes through module routing, resource limits, durable state, policy, and `RuntimeJournal`, but the reasoning behavior itself belongs to module-local agents.

1.3 Runtime Data Flow

Runtime data flow in **MADRE** is concurrent, governed, and module-centered. A local request enters through an access surface and is submitted to the kernel. The kernel records the request, applies the

relevant entry policy, and routes it to the Core Module by registration or to another registered module when routing evidence supports that decision.

Within the selected module, its Main Agent role may produce an immediate foreground response, create or continue delayed reasoning work, invoke internal actions, request inference through an allowed `ModelAction`, and reflect on recent outcomes. These activities are coordinated inside a module-owned agent and constrained by module policy, kernel policy, `ModelBinding`, resource assignment, durable state, `RuntimeJournal`, and recovery rules.

The following subsections describe the canonical request narrative, the governed model/action sequence, and the data-authority rules that prevent user input, source material, generated material, action results, or learning candidates from promoting themselves into authority-bearing state.

1.3.1 Canonical Request Narrative

The canonical user request begins when an access surface submits a local request to the kernel. The kernel does not interpret the request as a `KnowledgeRecord`, factual authority, or policy. It records a `RuntimeEvent`, applies entry constraints, and routes the work to the Core Module unless another module is already selected by session state, policy, or module registry evidence.

The receiving module evaluates the request through its module-local Main Agent role. If a configured foreground lane has enough safe local context, it may produce an immediate answer. If intent, scope, or risk is unclear, it asks for clarification. If the request requires evidence, stronger reasoning, internal actions, model inference, or broader context, the reasoning lane creates or continues a `ReasoningTask`. Reflection may run after the response or after delayed outcomes to propose a `LearningCandidate`. Knowledge-boundary change is separately represented as a `KnowledgeCandidate`.

Every significant step emits a `RuntimeEvent` through `RuntimeJournal`. Context bundles, actions, `RuntimeEvent` evidence and generated material objects, `PolicyDecision` outcomes, failures, corr ection, rollback, invalidation, supersession, and deletion/detachment remain inspectable through read-first surfaces.

1.3.2 Governed Model And Action Sequence

When module work requires executable behavior, the module invokes an internal typed action. The default action mechanism is local, typed, runtime-defined, policy-gated, and traceable. It is not an open plugin socket and not a network-exposed external action protocol.

A `ModelAction` is an `AgentAction` that uses inference. Before it runs, the module supplies a governed `ContextBundle`, an allowed `ModelBinding` and model-use profile, and the applicable `PolicyDecision`. An model backend executes inference without allowing a provider, template, or runtime format to define MADRE 's authority model. One execution is captured as an `RuntimeEvent` evidence; its generated material is an generated material.

Action results may be observations. generated material is generated material and is not a candidate by itself. It may inform a next reasoning step or undergo explicit handling into another record, but it does not authorize actions, create policy, become memory or knowledge, change routing or behavior, or promote learning. Each such transition requires its relevant authority boundary, verification path, `RuntimeEvent` record, and recovery expectation.

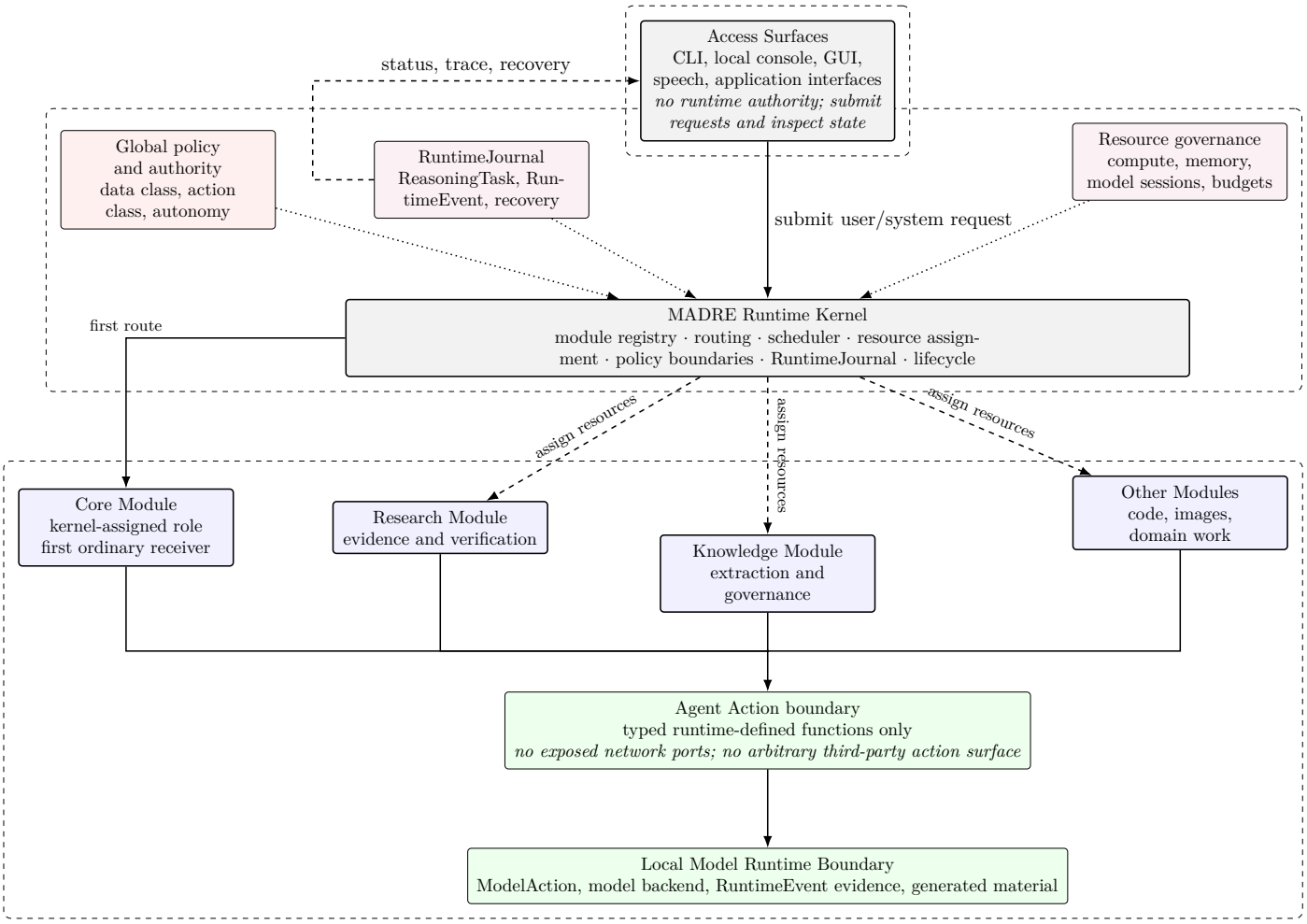


Figure 1.1: MADRE responsibility layers: kernel coordination, Core Module assignment, runtime journal, actions, and inference.

Figure 1.1 defines the top-level responsibility model. Access surfaces are intentionally outside runtime authority: they submit requests and inspect state, but they do not own policy, knowledge, routing, scheduling, or action-authority decisions. The kernel is the only system-level coordinator. It registers modules, assigns the Core Module, routes requests, assigns resources, applies global policy boundaries, records through `RuntimeJournal`, and maintains durable lifecycle state.

The module layer is the reasoning layer. The Core Module is shown first because ordinary interaction is routed there by registration, not because it is outside the module system. Research, knowledge, code, media, or domain modules are peers selected when the registry, policies, and current reasoning plan indicate a better fit. Internal actions and inference appear below the module layer because they are invoked by authorized module behavior, not by access surfaces or generated material directly.

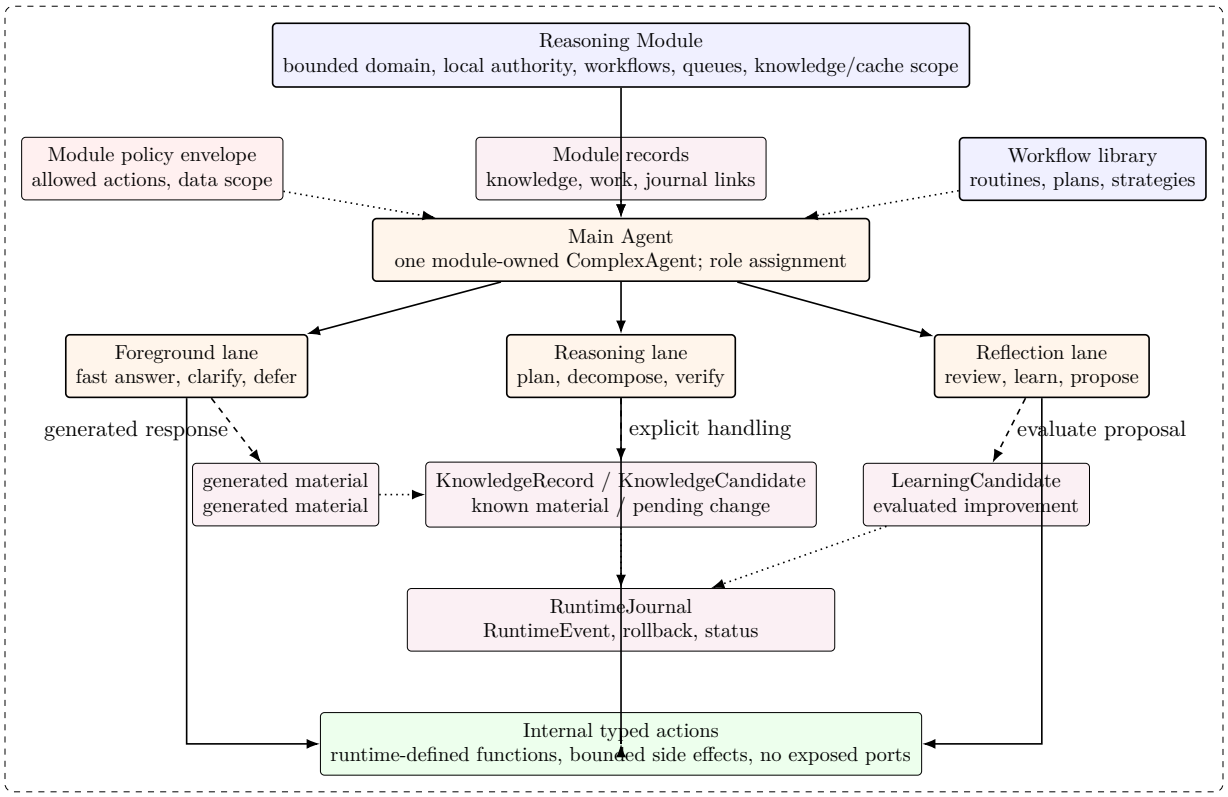


Figure 1.2: Reasoning module internal structure with its Main Agent using a ComplexAgent lane profile.

Figure 1.2 expands the internal structure of a reasoning module. A module owns a bounded domain, local policy envelope, records, workflow library, knowledge boundary, and concrete `MADREAgent` objects. One module-owned `SystemAgent` holds the Main Agent role; additional `ComplexAgent` objects may serve other module work without that role.

The foreground lane, reasoning lane, and reflection lane may operate concurrently. The foreground lane protects user continuity. The reasoning lane protects evidence, decomposition, verification, and stronger work. The reflection lane protects adaptation by evaluating a proposed `LearningCandidate` rather than silently rewriting runtime behavior. These lanes do not increase agent authority. They remain constrained by module policy, kernel policy, assigned resources, trace requirements, and promotion boundaries.

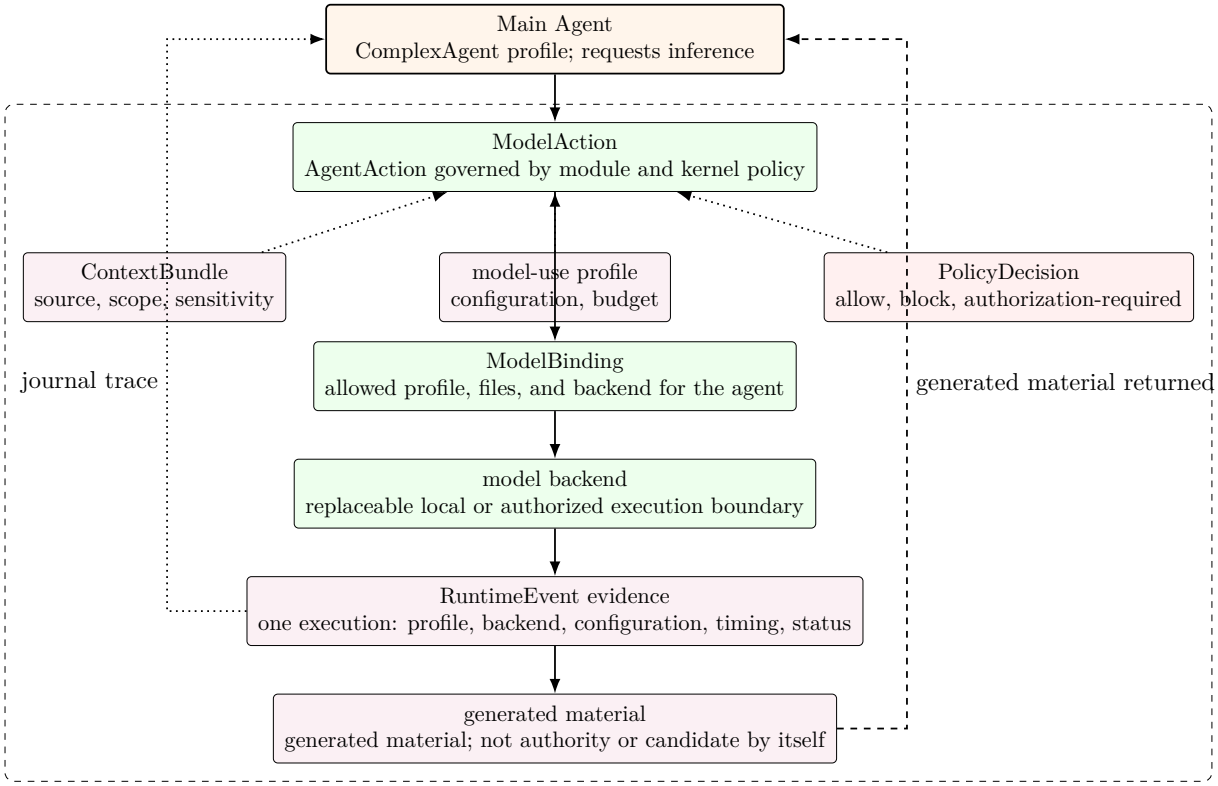


Figure 1.3: ModelAction inference path from governed execution through RuntimeEvent evidence to non-authoritative generated material.

Figure 1.3 describes inference as the governed `ModelAction` path rather than as an external server call or chat-completion product boundary. A module-owned `MADREAgent`, including one serving as Main Agent role, can request generation, critique, classification, or verification only under its allowed `ModelBinding` and an applicable `PolicyDecision`.

The model-use profile captures execution configuration, and the replaceable model backend executes the permitted inference. Every execution creates an `RuntimeEvent` evidence; the generated material attached to that execution is generated material. The Local Model Runtime Boundary remains useful as product wording for this controlled local or authorized execution boundary, not as a competing runtime class. generated material is not truth, policy, memory, knowledge, behavior, routing, authority, or a candidate by itself. Explicit module handling can retain it as retained interaction material, save appropriately classified material as `KnowledgeRecord`, evaluate it toward a `LearningCandidate`, or discard it, with the transition recorded in `RuntimeJournal`.

1.3.3 Data And Authority Rules

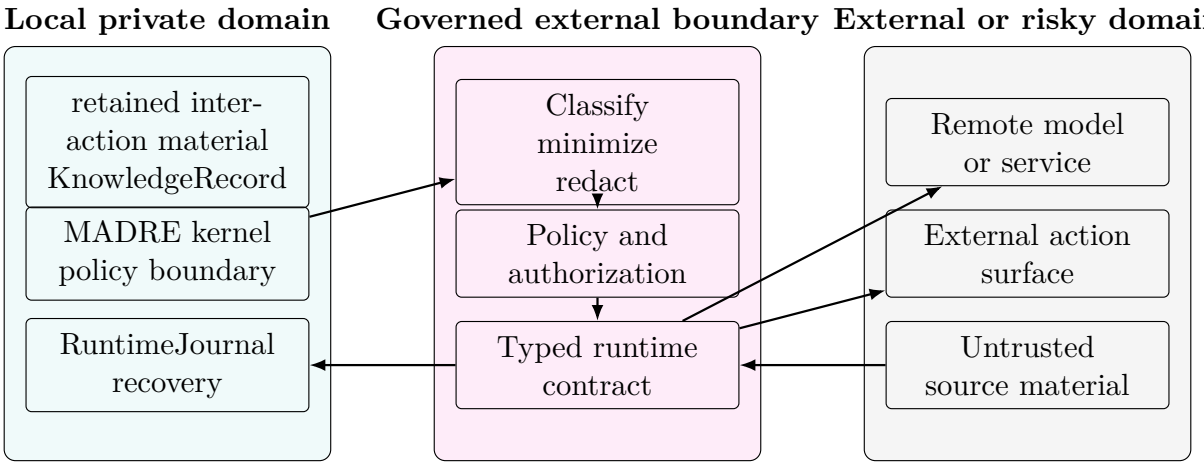
- User input enters as retained interaction material material when retained, not as policy, knowledge, truth, action authority, or model authority.
- Retrieved, extracted, or saved material becomes `KnowledgeRecord` only with provenance, scope, sensitivity, intended use, `truthLevel`, `truthAuthority`, correction, and lifecycle metadata; it is not factual truth by default.
- Context bundles are constructed, minimized, classified, scoped, and revocable before they reach an agent lane, internal action, or `ModelAction`.

- generated material is generated material from an `RuntimeEvent` evidence; it does not become an answer, policy decision, memory, knowledge, behavior, routing, authority, or candidate by itself.
- Internal action results enter as bounded observations. They become relied-on evidence only according to the action contract, verification path, and `RuntimeEvent` trace.
- External action surfaces, host commands, remote services, arbitrary filesystem mutation, credentials, and irreversible effects require stronger authority boundaries than ordinary internal actions.
- A `KnowledgeCandidate` represents only pending knowledge-boundary change; a `LearningCandidate` represents evaluated proposed module improvement and cannot directly rewrite active runtime behavior.

1.3.4 Trust Boundary Diagram

The trust boundary diagram distinguishes the ordinary local governed domain from optional external or risky domains. The default execution path remains inside the local governed domain: access surfaces, kernel coordination, reasoning modules, internal typed actions, local model runtime, `RuntimeJournal`, and recovery.

The external boundary exists for exceptional external actions: remote services, provider-backed models, host-command execution, arbitrary filesystem mutation, network access, untrusted source acquisition, or third-party integrations. These are not ordinary internal actions. They require classification, minimization, explicit policy authorization, recorded trace, and a recovery or blocking expectation appropriate to their risk class.



1.3.5 Access Surfaces

MADRE exposes access surfaces without allowing those surfaces to define the kernel. The `madre` CLI submits local requests and exposes status or result evidence. A local read-first inspection console may expose sessions, queued work, context bundles, policy outcomes, model evidence, internal action evidence, `KnowledgeCandidate`, `LearningCandidate`, `RuntimeJournal` traces, recovery, and runtime state. These surfaces operate under the same policy and authority model as every other runtime interaction.

The inspection console is not a privileged mutation path. Its transport is intentionally not fixed at architecture level. If a later implementation uses HTTP, sockets, embedded UI, or another mechanism, that choice must be justified by deployment and security decisions and must preserve local-first, read-first, policy-governed behavior.

Graphical, speech, product-specific, or integration surfaces remain access surfaces over the runtime rather than privileged paths around it.

Design Decision Traceability

Design decisions D-001 through D-011 provide traceability from product requirements to architecture obligations and validation evidence. They identify accepted product architecture decisions for governed delayed reasoning, durable work, context, inference, policy, recovery, bounded internal actions, local access, knowledge-boundary control, and evaluated learning promotion.

Decision	Product-level effect	Architecture boundary
D-001	Defines source-backed delayed-answer behavior as a core expression of governed runtime value.	Conversation remains connected to durable work and evidence.
D-002	Establishes behavior-first architectural validation as part of the product's engineering discipline.	Observable behavior anchors architecture claims.
D-003	Requires `ReasoningTask` lifecycle and projected inspection state.	`RuntimeJournal` and append-only `RuntimeEvent` preserve local traceability.
D-004	Requires conservative foreground triage.	Unsupported answers become clarification, policy block, or delayed work.
D-005	Requires governed context acceptance and rejection.	Context is sourced, classified, minimized, scoped, provenanced, and revocable.
D-006	Requires `ModelAction`, model backend, `RuntimeEvent` evidence, and non-authoritative generated material.	Models remain runtime functions, not authorities or candidate creators by generation alone.
D-007	Requires explicit `PolicyDecision` before action execution.	Action execution follows policy rather than model instruction.
D-008	Requires recovery as a recorded semantic transition.	Correction, rollback, invalidation, supersession, deletion, and detachment have distinguishable `RuntimeEvent` meaning.

Decision	Product-level effect	Architecture boundary
D-009	Defines bounded internal actions as the low-risk executable reference.	Action execution is policy-gated, traceable, and recoverability-aware.
D-010	Defines CLI request/status interaction and local read-first console inspection as canonical local access surfaces.	Access surfaces expose the kernel without becoming runtime authority.
D-011	Distinguishes pending `KnowledgeCandidate` boundary changes from evaluated `LearningCandidate` module improvements.	Known-material storage is not learning; proposed evolution remains inspectable until promoted by authority.

Architecture Viewpoints

3.1 Viewpoint Catalogue

Viewpoint	Primary concern	MADRE content
System context	Runtime boundary and environment	User, local device, filesystem, operating system, local models, optional external integrations, internal actions, and untrusted sources.
Logical	Responsibility separation	Kernel, Core Module, module-owned agents, scheduler, context/knowledge, Local Model Runtime Boundary, Agent Action boundary, `PolicyDecision`, `RuntimeJournal`, learning, and recovery.
Runtime/process	Work lifecycle	User turn, triage, `ReasoningTask`, context bundle, scheduling, `RuntimeEvent` evidence, result delivery, `RuntimeEvent`, and recovery.
Data/knowledge	Data ownership and provenance	retained interaction material, `KnowledgeRecord`, `KnowledgeCandidate`, `LearningCandidate`, truth metadata, retention, correction, and deletion.
Security/trust	Boundary enforcement	Local domain, governed external boundary, external/risky domain, prompt injection, remote transfer, credential exposure, and action side effects.

Viewpoint	Primary concern	MADRE content
Deployment	Local operation	Ordinary device profile, local storage, model availability, backup, migration, diagnostics, and optional remote configuration.
Extension	Developer evolution	`ModelBinding`, model-use profile, model backend, action contracts, workflows, projections, context policies, and interface surfaces.
UX	User-visible control	Conversation, clarification, queue state, progress, authorization, failure, completion, correction, and rollback.

3.2 Viewpoint Design Focus

Viewpoint	Architecture focus	Authority boundary
System context	Local user, local device, CLI, local read-first console, local sources, and optional remote integrations.	Local authority remains explicit across every access surface.
Logical	Kernel coordination over routing, delayed work, context, bounded action use, `PolicyDecision`, `RuntimeJournal`, and recovery.	Runtime authority is not delegated to interfaces, models, actions, or generated material.
Runtime/process	Governed lifecycle from local request to inspection and recovery.	`ReasoningTask` transitions remain recorded, inspectable, and recoverable.
Data/knowledge	Governed context and explicit knowledge/learning types.	Source material, retained interaction material, `KnowledgeRecord`, `KnowledgeCandidate`, and `LearningCandidate` remain distinguishable.
Security/trust	Source-as-data, model-as-function, policy before action, and bounded internal action authority.	Action execution follows declared policy outcomes.

Viewpoint	Architecture focus	Authority boundary
Observability/recovery	Local status, `RuntimeJournal`, verification, policy, knowledge/learning transition, and recovery evidence.	Read-first inspection must not become a privileged mutation path.
Extension	Inference backends, actions, workflows, and learning systems inherit the defined boundaries.	Extension preserves kernel authority and product traceability.